

Smart Gate Access Control System (GP2)

A smart university campus entry system designed to automate identity verification, enhance security, and streamline student access using computer vision and pretrained deep learning embeddings.

| Project Overview

Our project introduces a smart gate system for universities to make campus entry safer and more organized. The system uses a camera at the gate connected to a central database containing photos and information of all registered students, staff, admins, and security personnel.

When people pass by the camera, it provides a live video feed to the university security guards.

- **Authorized Access:** If a registered individual passes, the system recognizes them, momentarily displays a green bounding box around their face on the screen, logs the entry, and allows them to enter.
- **Unauthorized Access / Recognition Errors:** If an unregistered person passes—or if the system fails to recognize a registered student for any reason—it automatically captures a screenshot of their face, displays a red bounding box, and triggers an audio alarm on the security monitor.

Manual Verification Fallback

To ensure smooth operations under all real-world conditions, the system relies on a manual backup process whenever it fails to recognize someone:

1. When a red alert is triggered, the security guard asks the individual for their University ID number.
2. The guard enters this number into the system interface.
3. If the individual is a registered student or staff member, their photo, role, and information pop up on the screen immediately so the guard can verify their identity and allow them in.
4. If the person is an outsider with no record in the database, the security personnel will handle the situation according to university regulations.

The system also includes a dedicated management portal for the admission and registration department to easily add new student records or remove old ones.

| Technical Architecture & AI Approach

AI & Computer Vision Method

- **Face Recognition Model:** Inference-only pipeline using the DeepFace framework with the pretrained VGG-Face model architecture. The repository operates 100% on inference and does not include a custom training pipeline.
- **Face Detection:** Handled primarily via the DeepFace detector backend (OpenCV-based), with an OpenCV Haar Cascade classifier fallback if DeepFace fails.

- **Embedding Extraction:** Facial features are extracted as 4,096-dimensional vectors using `DeepFace.represent()` with default inference settings. Each embedding takes approximately 36 KB of storage.
- **Matching Strategy:** Verification is performed via cosine-distance comparison against stored embeddings using a configurable distance threshold (`GP2_MATCH_THRESHOLD`).

Implementation & Tech Stack

- **Language:** Python 3
- **AI Frameworks and Libraries:** DeepFace, OpenCV (cv2), NumPy, Pickle
- **Backend:** Flask + Flask-SocketIO for real-time data and alert broadcasting
- **Frontend:** HTML/JS templates with a native Socket.IO client (index.html)
- **Database:** SQLite (university_runtime.db) optimized with PRAGMA settings, storing metadata and serialized embedding vectors as BLOBs
- **Storage Structure:**
 - `students_images/` : Contains source enrollment images organized by subdirectory roles:
 - `students_images/student/` (Role: Student)
 - `students_images/staff/` (Role: Staff)
 - `students_images/admin/` (Role: Admin)
 - `students_images/security/` (Role: Security)
 - `captures/` : Directory where unauthorized or unverified screenshots are saved for security review

| Data Description & Preprocessing

Dataset Specifications

- **Dataset Source:** University registration and admission facial photographs collected with user consent. Supported formats include JPG, PNG, BMP, and WebP.
- **Dataset Size Scalability:**
 - Current Demo Status: ~10 students, with a database size of ~8 KB (test data only).
 - Target Baseline: 1,000 students (~35 MB database size).
 - Recommended Capacity: 2,000 students (~70 MB database size).
 - Maximum Capacity: 5,000 students (~176 MB database size).
- **Data Split:** Currently 100% inference-only using pretrained models. If a training pipeline is added in the future, the proposed split is 70% Training, 15% Validation, and 15% Test.

Preprocessing and Quality Validation

1. **Face Detection and Cropping:** The facial region is extracted from the original image.

2. **Minimum Size Validation:** Faces must be at least 90x90 pixels. Images with smaller facial regions are automatically skipped and logged.
3. **Normalization:** Color space standardization is applied for model input. Histogram equalization is utilized as a fallback mechanism for lighting adjustment.
4. **Labeling and Role Extraction:**
 - Student IDs and names are extracted automatically from filenames formatted as `ID_Name.jpg` (e.g., `2019001_Ahmed_Mohammed.jpg` extracts ID: 2019001, Name: Ahmed Mohammed).
 - Roles are assigned automatically based on the parent folder name under `students_images/`.

System Outputs & Access Decision Labels

- **Allowed:** Verified student or staff member. Emits a `security_event` via Socket.IO, triggers a green bounding box, and creates an entry in `access_logs`.
- **Unknown / Unverified:** Face not found or not recognized by the database. Emits an alert via Socket.IO, triggers a red bounding box with a sound alarm, prompts manual verification via ID number, and saves a screenshot to the `captures/` directory.

Database Architecture

The SQLite database (`university_runtime.db`) consists of the following primary tables:

- `students` : Stores student/staff metadata, text fields, and pickled embedding BLOB fields.
- `access_logs` : Tracks all real-time access attempts, timestamps, and system decisions.
- `graduated_students` : Holds archives for alumni records.
- `admission_students` : Handles staging data for new entries awaiting system synchronization.

Update Mechanism: When new enrollment images are added to the directories, the system automatically triggers the re-extraction of embeddings. No periodic retraining is conducted since the core system relies on a fixed pretrained model.

Note on Current Limitations and Scope: This codebase is optimized for real-time inference. Advanced features like model quantization/export (ONNX/TFLite), FAISS/ANN accelerated search, and dedicated model training scripts are currently not implemented.

Configuration & Environment

The application behavior can be customized using the following environment variables:

- `GP2_MATCH_THRESHOLD` : Adjusts the cosine-distance threshold for face matching.
- `GP2_DATABASE_PATH` : Specifies the path to the SQLite database.

- `GP2_CAMERA_INDEX` : Configures the hardware camera index for OpenCV video capture.
- `GP2_FRAME_SKIP_INTERVAL` : Sets the frame-skipping rate to optimize processing performance on hardware like Raspberry Pi.